Hello World!

In this lesson, you will get acquainted with the Hello World program and a basic introduction to Java.

WE'LL COVER THE FOLLOWING

- Hello World: Example
- `Java` as a structural language
- ♨️ New features
- 🙅 Bad practices

Hello World: Example

The first program that most aspiring programmers write is the classic **"Hello World"** program. The *purpose* of this program is to display the text **"Hello World!"** to the user.

The **"Hello World"** example is somewhat famous as it is often the first program presented when introducing a programming language.

Try running the code below!

```
class HelloWorld {
    public static void main(String args[]) {
        System.out.println("Hello World!");
    }
}
```

> **Did you know?**
>
> The `System.out.println()` displays text in the current line of the console and then move the cursor to the beginning of the next line.

`Java` as a structural language

Before discussing the particulars, it is useful to think of a computer program in terms of both its **structure** and its **meaning** simultaneously.

A `Java` program is structured in a specific and particular manner. `Java` is a language and therefore has grammar similar to a spoken language like *English*. The grammar of computer languages is usually much, much simpler than spoken languages but comes with the disadvantage of having stricter rules. Applying this structure or grammar to the language is what allows the computer to understand the program and what it is supposed to do.

The overall program has a **structure**, but it is also important to understand the purpose of part of that **structure**. By analogy, a textbook can be split into sections, chapters, paragraphs, sentences, and words (the structure), but it is also necessary to understand the overall meaning of the words, the sentences, and chapters to fully understand the content of the textbook. You can think of this as the semantics of the program.

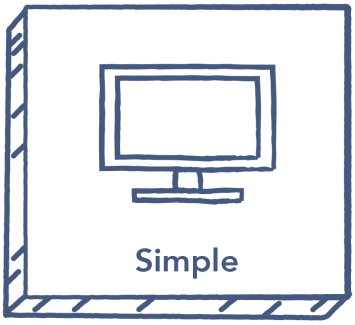
A line-by-line analysis of the program should give a better idea of both the **structure** and meaning of the classic **"Hello World"** program. Let's take a look at it in the upcoming lessons.

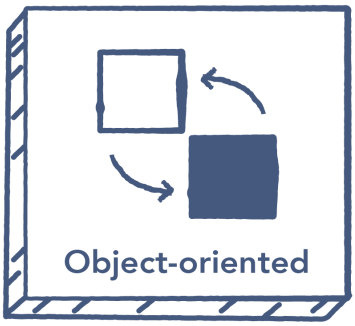
New features

The new features and upgrades included in Java changed the face of the programming environment and gave a new definition to *Object-Oriented Programming* (OOP in short). But unlike its predecessors, Java needed to be bundled with standard functionality and be independent of the host platform.

The primary goals in the creation of the Java language:

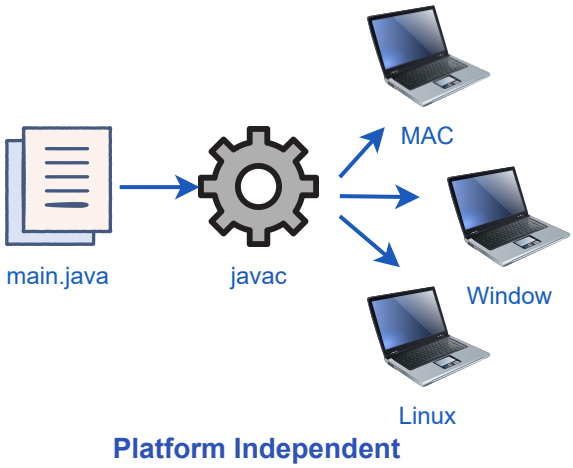
- It is **Simple & Portable**: Memory Management using Pointers is not allowed.





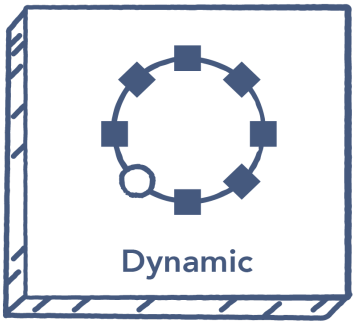
- It is **Object-Oriented**.

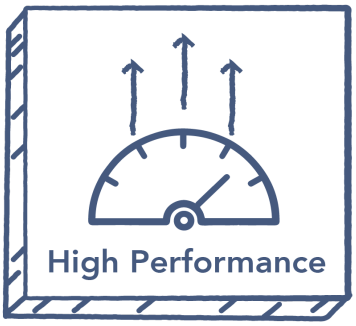
- It is **Independent** of the host platform.



- It is **Secured & Dynamic**: Designed to execute code from remote sources securely.

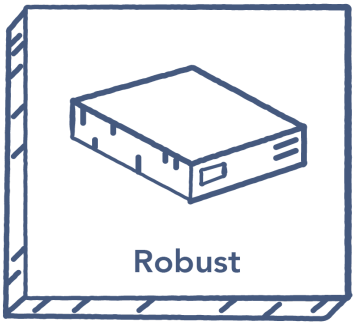
- It contains language facilities and libraries for **Networking.**

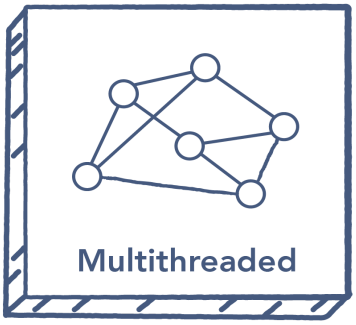




- **High Performance:** With the use of JIT (*Just-In-Time*) compilers, Java achieves high performance through the use of **byte-code** that can be easily translated into native machine code.

- It is **Robust**: Java has its own strong **memory management** system. This helps to eliminate errors as it checks the code during compile and runtime.





- Java is **Multithreaded:** It supports multiple executions of threads (i.e., *lightweight processes*), including a set of synchronization primitives.

The Java language introduces some new features that did not exist in other

The Java language introduces some new features that did not exist in other languages like C and C++.

🤐 Bad practices

Over the years, some features in C/C++ programming became abused by the programmers. Although the language allows it, it was known as bad practices. So the creators of Java have disabled them:

- Use of Pointers
- Operator overloading
- Multiple inheritance
- Friend classes (access another object's private members)
- Restrictions of explicit type casting (related to memory management)

Let's give you a detailed breakdown of the above code in the next lesson!

Code Explanation

In this lesson, we will explore how the "Hello World!" program works in Java.

WE'LL COVER THE FOLLOWING

- The given code
- Detailed explanation
  - `class HelloWorld`
  - `void main()`
  - `{}`
  - Print statement
  - Semicolons

The given code

```
class HelloWorld {
    public static void main(String args[]) {
        System.out.println("Hello World!");
    }
}
```

Detailed explanation

Now let's look at the **Hello World!** sequentially.

`class HelloWorld`

It is written as:

```
class HelloWorld
```

Java is an object-oriented programming language. Therefore, every line that needs to be executed should be inside a class. This line declares a class `HelloWorld`.

- **Declarations and Definitions:** A class declaration provides the name and visibility of a class. In our example, `class HelloWorld` is the class declaration. It consists (in this case) of one keyword, `class` followed by the identifier `HelloWorld`.
- The `class` **declaration** is then followed by a block (surrounded by curly braces `{}`), which provides the class's definition.

This means that we are defining a class named `HelloWorld`. Other classes, or in our case, the command line, can refer to the class by this name.

`void main()`

The `main` is the point from where all the Java programs start its execution. In order to execute your program, you must follow the valid signature of `main()` method which is given below:

```
public static void main( String args[] )
```

Don't worry about the details of classes, objects, and the signature of the `main()` method. We will cover it in detail in the upcoming chapters. For now, just remember that we will always write our code inside the class, and this class must contain the `main` method.

> **Note:** In object-oriented programming languages, the program consists of objects. Objects are comprised of methods and the associated data. All objects of the same kind are instances of the same class, which provides a template for all of its objects. For a Java program to run, it must have at least one object, which must define a method called `main()`.

{ }

A block of code is defined with the `{ }` tokens. `{` signifies the start of a block of code, and `}` signifies the end.

> **NOTE:** The `{ }` tokens have other uses as well.

Print statement

```
System.out.println( "Hello World!" );
```

- `System.out.println( "Hello World!" );`

  Gives processor the command to `output` the literal specified in the double quotes ("") and adds a new line to the console. In this case, **Hello World!** is printed.

> **Did you know?** `System.out.print()` prints the content and does not add the new line.

Semicolons

Statements in Java must be terminated with a **semicolon**,

- Just as sentences in English must be terminated with a period.
- Just as sentences in English can span several lines, so can the statements in Java.

In fact, you can use as many spaces and new lines between the words of a Java program as you wish to beautify your code just as spaces are used to justify the text printed on the pages of a book.

Now that we have learned the basics of our code. Let's learn how to compile our code in Java in the next lesson!

Compiling Your First Java Program!

In this lesson, an explanation of how to compile your code in Java, which compilers to use and the most suitable ones for different Operating Systems, is provided.

WE'LL COVER THE FOLLOWING

- What does the compiler do?
  - **Did you Know?**
- Running the compiler
- Compilation procedure

For the computer to execute the *Hello World* code you have written, it first needs to be compiled by a Java compiler. The compiler translates the computer code written in a particular programming language into the machine code.

What does the compiler do?

In very broad terms:

> **Did you Know?**
>
> The compiler is a translator that acts as an intermediary between the programmer and the CPU on the computer.

A high-level language like Java is actually a sort of '*compromise*' language between the native language of the CPU (generally referred to as machine language) and the native language of the programmer (say English).

Computers do not natively understand human languages. Therefore, the compiler is reading the code written by the programmer and translating it into machine language code that the computer can execute directly.

In `Java`, programs are **not** compiled into **executable files**; they are compiled into *bytecode*, which the JVM (Java Virtual Machine) then executes at runtime. Java source code is compiled into *bytecode* when we use the javac compiler. The *bytecode* gets saved on the disk with the file extension `.class`. When the program is to be run, the bytecode is converted using the just-in-time (JIT) compiler. The result is machine code, which is then fed to the memory and is executed.

In short, we follow the following two steps to execute the Java program:

1. Java programs need to be compiled to **bytecode**.
2. Then we need to convert the **bytecode** into **machine code**.

The Java classes/bytecode are compiled to machine code and loaded into memory by the JVM when needed the first time. This is different from other languages like C/C++, where programs are to be compiled to machine code and linked to create an executable file before it can be executed.

Running the compiler

The code needs to be compiled with a compiler to finish the process.

What if you don't have one?

Well, the good news is, there are *several* good compilers that are available for free.

- The Eclipse has various versions available for most systems and does a good job of implementing the **Java** standard libraries.
- The NetBeans has complete support for Java used for multiple purposes, including **WebApps** and **Web-Services**. It also provides Git Repositories to its users.
- Some Mac Users prefer using Xcode for running Java on their machine.

There are multiple compilers for one language, and picking out one is totally a personal preference.

Compilation procedure

We are providing a Java execution environment in our course, so you don't have to worry about this. But, if you prefer to install the environment on your own machine for other uses, you can follow the instructions given in this link.

We hope you've successfully installed and run your first Java program.

That's it for now, let's play around with some print statements in the next lesson to get *'you'* used to it.

Challenge: Play Around With 'print' Statement

In this challenge, we will display particular text on the console.

WE'LL COVER THE FOLLOWING

- Try yourself

Try yourself

Print this line using `System.out.print()`:

The movie quote is: My name is Bond. James Bond!

> **Important Note:** When you write the above line, make sure there is only a single space between each word, and no space between `is:` . Otherwise, your code won't pass the test case.

It would be better if you try using the `System.out.println()` statement by yourself first and then see the solution.

```
class HelloWorld {
    public static void print() {
        // write your code here
        System.out.println("write your code here");
    }
}
```

For a step by step break down of the solution, let's go to the solution review in the upcoming lesson.

Solution Review: Play Around With 'print' Statement

In this review, a quick analysis of the Print Statement Challenge is provided.

WE'LL COVER THE FOLLOWING

- Given solution
- Explanation

Given solution

```
class HelloWorld {
    public static void main( String args[] ) {
       System.out.println("The movie quote is: My name is Bond. James Bond!");
    }
}
```

Explanation

The above program is quite self-explanatory. We put the required piece of text in double quotation marks inside `System.out.println()`, and it will automatically display the required text on the console.

Let's wrap up this chapter by solving a quiz in the upcoming lesson.